Algorithms (Recap)

Algorithm

An **algorithm** is any well-defined computational procedure that

- takes as input a value or a set of values,
- outputs some value or a set of values,
- in a finite amount of time.

Algorithm

An **algorithm** is any well-defined computational procedure that

- takes as input a value or a set of values,
- outputs some value or a set of values,
- in a finite amount of time.

Each step must be clear and unambiguous

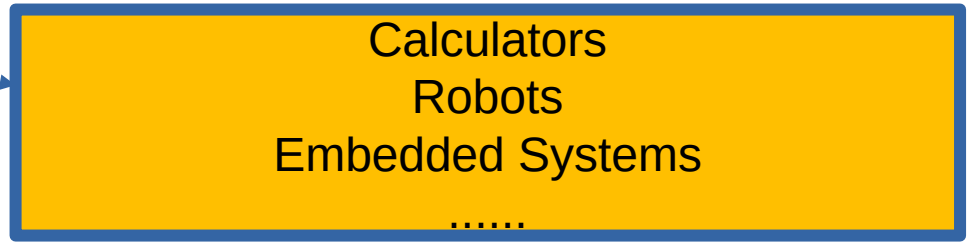
It finishes in a finite number of steps (it halts)



* well-defined (unambiguous)
* halt

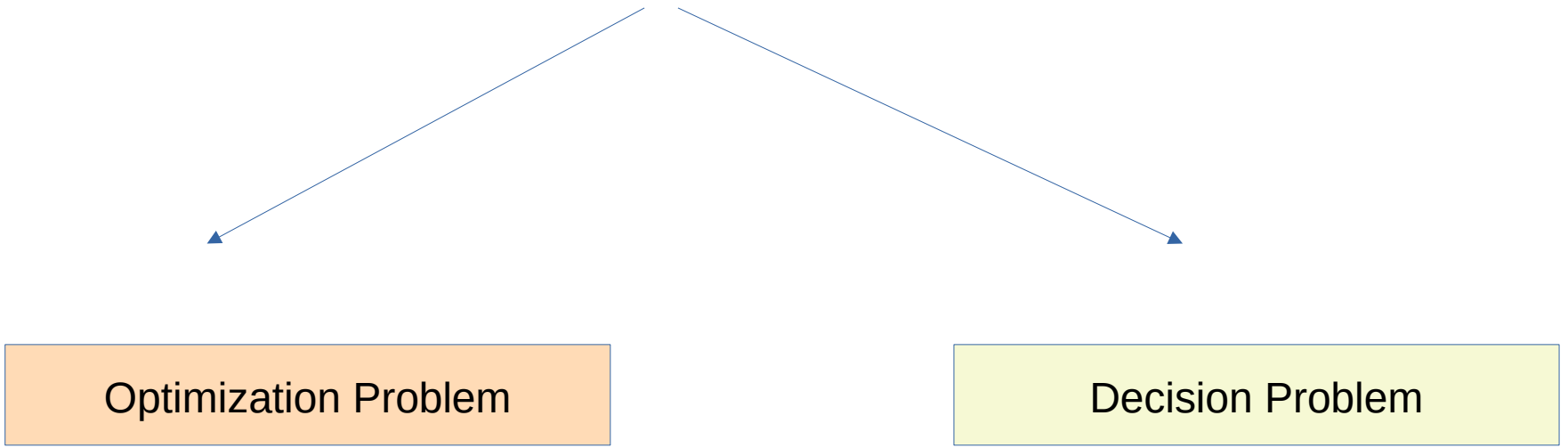
Computational Problem

- A problem that can be solved using an **algorithm** by a computer or any computational device.

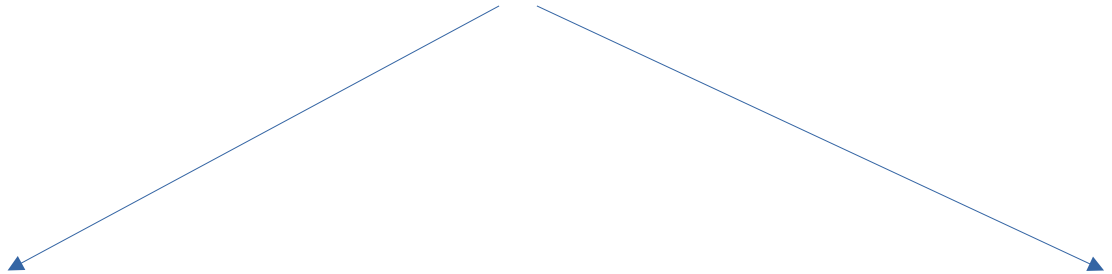


Algorithm is a tool for solving a well-specified computation problem.

Computational Problem



Computational Problem



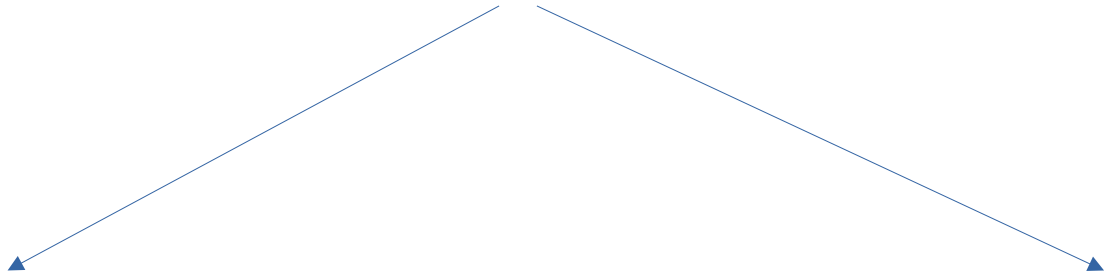
Optimization Problem

Decision Problem

“Optimize the value”

“YES / NO”

Computational Problem



Optimization Problem

Decision Problem

"Optimize the value"

Asks for the best possible solution according to some criterion.

"YES / NO"

Asks whether a condition is true (YES/NO)

Computational Problem

- Input: A sequence of numbers <a1, a2, ….. , an>
- Output: A permutation <b1, b2, ….. , bn> of the input sequence such that b1 ≤ b2 ≤ ….. ≤ bn.

Can you model this as a decision problem?

Computational Problem

- Input: A sequence of numbers <a1, a2, ….. , an>
- Output: A permutation <b1, b2, ….. , bn> of the input sequence such that b1 ≤ b2 ≤ ….. ≤ bn.

- **DECISION:** Does there exist a permutation <b1, b2, ….. , bn> such that b1 ≤ b2 ≤ ….. ≤ bn ?

Can you model this as an optimization problem?

Computational Problem

- Input: A sequence of numbers <a1, a2, ….. , an>
- Output: A permutation <b1, b2, ….. , bn> of the input sequence such that b1 ≤ b2 ≤ ….. ≤ bn.

OPTIMIZATION:

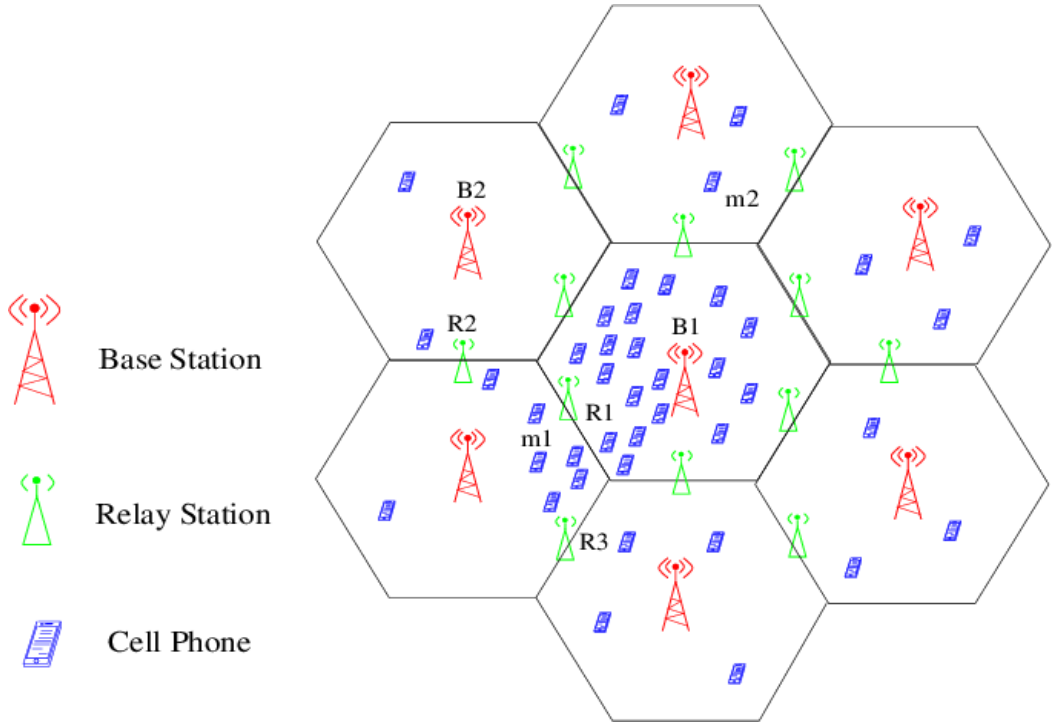
minimize the number of order violations (inversions),

$$\text{minimize} \sum_{i<j} \mathbf{1}[b_i > b_j]$$

where 1 is 1 if condition is true and 0 otherwise.

Computational Problem

- Input: A set of base stations and clients.
- Output: The minimum number of frequencies required to be assigned to base stations for proper communication.



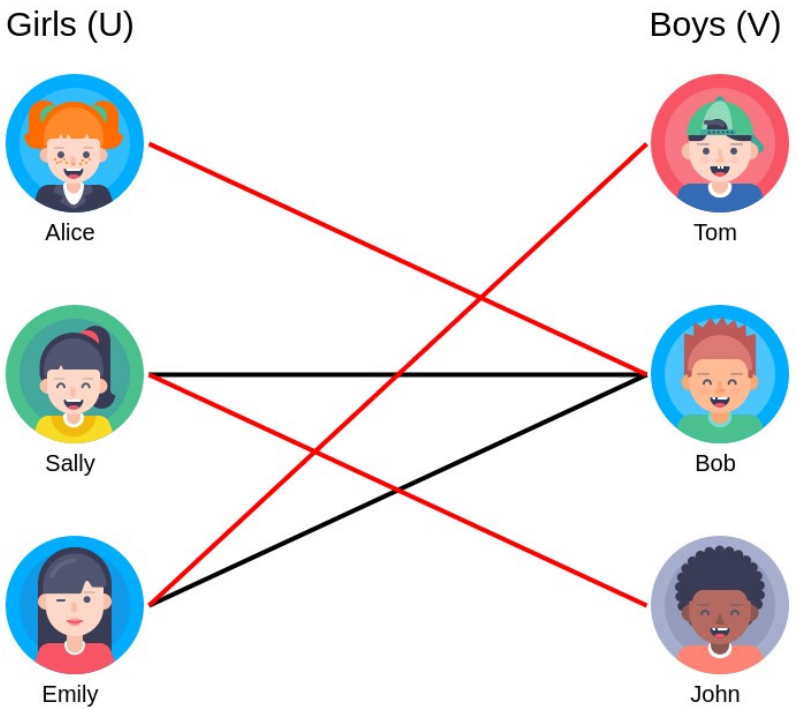
Computational Problem

- Input: Map
- Output: The minimum number of colors required to color the states such that
  - states that share a boundary are
  - colored distinctly.



Computational Problem

- Input: A set of Girls and Boys, and their compatibilities.
- Output: Find the maximum number of pairs that are compatible.

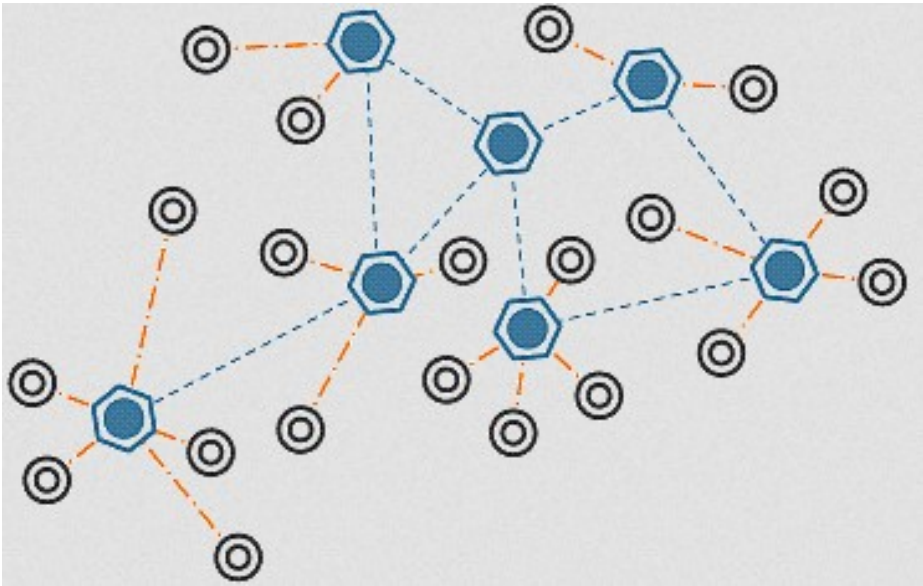


Computational Problem

- Input: Potential surveillance points, and the underlying graph.
- Output: The minimum number of cameras to be installed such that each region in covered.

Computational Problem

- Input: Potential surveillance points, and the underlying graph.
- Output: The minimum number of cameras to be installed such that each region in covered.



Questions

Is GRAPH COLORING polynomial time solvable?

The graph coloring problem is the problem of assigning colors to the vertices of a graph such that no two adjacent vertices have the same color, using the minimum number of colors.

Questions

Is GRAPH COLORING polynomial time solvable?

The graph coloring problem is the problem of assigning colors to the vertices of a graph such that no two adjacent vertices have the same color, using the minimum number of colors.

Is MAP COLORING polynomial time solvable?



Questions

Is GRAPH COLORING polynomial time solvable?

The graph coloring problem is the problem of assigning colors to the vertices of a graph such that no two adjacent vertices have the same color, using the minimum number of colors.

Is MAP COLORING polynomial time solvable?

- You can form a planar graph from the map.
- Planar graph is 4-colorable.



Questions

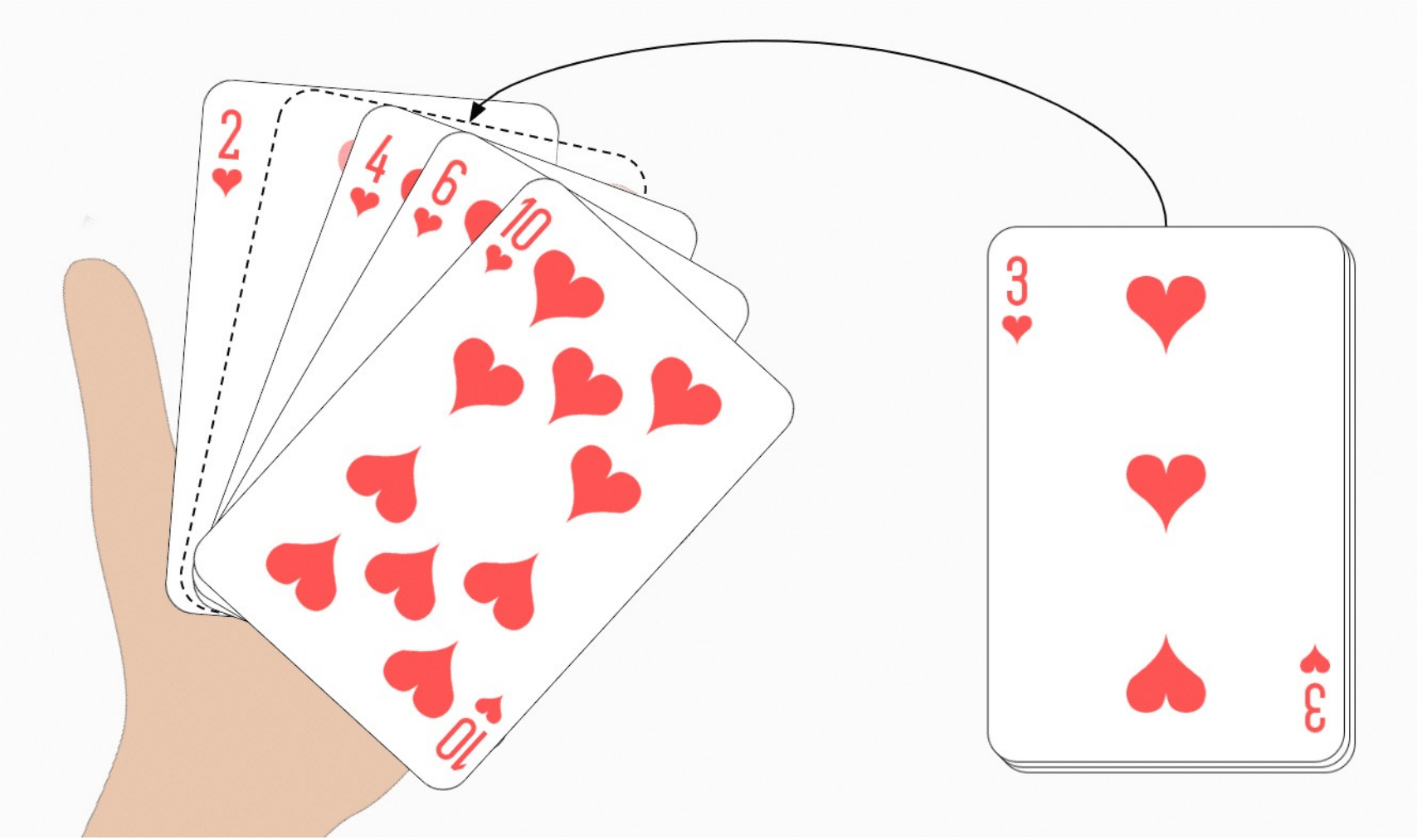
Can you use decision version to find the optimum value, for GRAPH COLORING?

Decision Version:

Input: graph G and an integer k.

Question: Does there exists a proper coloring of G using k colors?

INSERTION-SORT



EXAMPLE: INSERTION-SORT

```
INSERTION-SORT(A, n)
1  for i = 2 to n
2      key = A[i]
3      // Insert A[i] into the sorted subarray A[1 : i − 1].
4      j = i − 1
5      while j > 0 and A[j] > key
6          A[j + 1] = A[j]
7          j = j − 1
8      A[j + 1] = key
```

INSERTION-SORT

Will you get the same run time of the code when you run multiple times?

INSERTION-SORT

NO

Will you get the same run time of the code when you run multiple times?

So, we better analyse the algorithm itself

- How many times each line of pseudocode is executed?
- How long each line takes to run?

INSERTION-SORT

NO

Will you get the same run time of the code when you run multiple times?

So, we better analyse the algorithm itself

- How many times each line of pseudocode is executed?
- How long each line takes to run?

Find the dominating factor (to analyze other algorithms)

Design and Analysis of Algorithms

Efficient Algorithms!

- Input: Consider two array A and B each of size N.
- Question: Find all common entries between A and B and output to C.

Efficient Algorithms!

- Input: Consider two array A and B each of size N.
- Question: Find all common entries between A and B and output to C.

Running time: $O(N^2)$

```
begin
    k ← 1

    for i ← 1 to N do
        for j ← 1 to N do
            if A[i] = B[j] then
                C[k] ← A[i]
                k ← k + 1
                break
            end if
        end for
    end for

    return C
end
```

Efficient Algorithms!

- Input: Consider two array A and B each of size N.
- Question: Find all common entries between A and B and output to C.

Running time: $O(N^2)$

How many instructions does a computer execute in one second?

```
begin
    k ← 1

    for i ← 1 to N do
        for j ← 1 to N do
            if A[i] = B[j] then
                C[k] ← A[i]
                k ← k + 1
                break
            end if
        end for
    end for

    return C
end
```

Efficient Algorithms!

- Input: Consider two array A and B each of size N.
- Question: Find all common entries between A and B and output to C.

Running time: $O(N^2)$

How many instructions does a computer execute in one second?

Say, **10 billion instructions per second**

```
begin
    k ← 1

    for i ← 1 to N do
        for j ← 1 to N do
            if A[i] = B[j] then
                C[k] ← A[i]
                k ← k + 1
                break
            end if
        end for
    end for

    return C
end
```

Efficient Algorithms!

- Input: Consider two array A and B each of size N.
- Question: Find all common entries between A and B and output to C.

Running time: $O(N^2)$

How many instructions does a computer execute in one second?

Say, **10 billion instructions per second**

Suppose N ~ 10^{10}, how much time does your computer take?

```
begin
    k ← 1

    for i ← 1 to N do
        for j ← 1 to N do
            if A[i] = B[j] then
                C[k] ← A[i]
                k ← k + 1
                break
            end if
        end for
    end for

    return C
end
```

Efficient Algorithms!

- Input: Consider two array A and B each of size N.
- Question: Find all common entries between A and B and output to C.

Running time: $O(N^2)$

How many instructions does a computer execute in one second?

Say, **10 billion instructions per second**

Suppose N ~ 10^{10}, how much time does your computer take?

$N^2=10^{20}$ instructions

```
begin
    k ← 1

    for i ← 1 to N do
        for j ← 1 to N do
            if A[i] = B[j] then
                C[k] ← A[i]
                k ← k + 1
                break
            end if
        end for
    end for

    return C
end
```

Efficient Algorithms!

- Input: Consider two array A and B each of size N.
- Question: Find all common entries between A and B and output to C.

Running time: $O(N^2)$

How many instructions does a computer execute in one second?

Say, **10 billion instructions per second**

Suppose N ~ 10^{10}, how much time does your computer take?

$N^2=10^{20}$ instructions **10^{10} seconds ~**

```
begin
    k ← 1

    for i ← 1 to N do
        for j ← 1 to N do
            if A[i] = B[j] then
                C[k] ← A[i]
                k ← k + 1
                break
            end if
        end for
    end for

    return C
end
```

Efficient Algorithms!

- Input: Consider two array A and B each of size N.
- Question: Find all common entries between A and B and output to C.

Running time: $O(N^2)$

How many instructions does a computer execute in one second?

Say, **10 billion instructions per second**

Suppose N ~ 10^{10}, how much time does your computer take?

$N^2=10^{20}$ instructions **10^{10} seconds ~ 330 years**

```
begin
    k ← 1

    for i ← 1 to N do
        for j ← 1 to N do
            if A[i] = B[j] then
                C[k] ← A[i]
                k ← k + 1
                break
            end if
        end for
    end for

    return C
end
```

Efficient Algorithms!

- Input: Consider two array A and B each of size N.
- Question: Find all common entries between A and B and output to C.

Suppose that the arrays A and B are sorted. Can we do better?

Efficient Algorithms!

- Input: Consider two array A and B each of size N.
- Question: Find all common entries between A and B, and output to C.

Sorted arrays A and B

```
i ← 1, j ← 1
while i ≤ N and j ≤ N do
    if A[i] = B[j] then
        output A[i]
        i ← i + 1
        j ← j + 1
    else if A[i] < B[j] then
        i ← i + 1
    else
        j ← j + 1
end while
```

Efficient Algorithms!

- Input: Consider two array A and B each of size N.
- Question: Find all common entries between A and B, and output to C.

Running time: O(N)

Sorted arrays A and B

Total running time:

```
i ← 1, j ← 1
while i ≤ N and j ≤ N do
    if A[i] = B[j] then
        output A[i]
        i ← i + 1
        j ← j + 1
    else if A[i] < B[j] then
        i ← i + 1
    else
        j ← j + 1
end while
```

Efficient Algorithms!

- Input: Consider two array A and B each of size N.
- Question: Find all common entries between A and B, and output to C.

Running time: O(N)

Sorted arrays A and B

Total running time: Time taken to sort the arrays
+
Time taken to find common entries

= O(N log N)

```
i ← 1, j ← 1
while i ≤ N and j ≤ N do
    if A[i] = B[j] then
        output A[i]
        i ← i + 1
        j ← j + 1
    else if A[i] < B[j] then
        i ← i + 1
    else
        j ← j + 1
end while
```

Efficient Algorithms!

- Input: Consider two array A and B each of size N.
- Question: Find all common entries between A and B, and output to C.

Running time: O(N)

Sorted arrays A and B

Total running time: Time taken to sort the arrays
+
Time taken to find common entries

= O(N log N)

Suppose N ~ 10^{10}, how much time does your computer take?

N log N ~ instructions

```
i ← 1, j ← 1
while i ≤ N and j ≤ N do
    if A[i] = B[j] then
        output A[i]
        i ← i + 1
        j ← j + 1
    else if A[i] < B[j] then
        i ← i + 1
    else
        j ← j + 1
end while
```

Efficient Algorithms!

- Input: Consider two array A and B each of size N.
- Question: Find all common entries between A and B, and output to C.

Running time: O(N)

Sorted arrays A and B

Total running time: Time taken to sort the arrays
+
Time taken to find common entries

= O(N log N)

Suppose N ~ 10^{10}, how much time does your computer take?

N log N ~ 40 * 10^{10} instructions

```
i ← 1, j ← 1
while i ≤ N and j ≤ N do
    if A[i] = B[j] then
        output A[i]
        i ← i + 1
        j ← j + 1
    else if A[i] < B[j] then
        i ← i + 1
    else
        j ← j + 1
end while
```

Efficient Algorithms!

- Input: Consider two array A and B each of size N.
- Question: Find all common entries between A and B, and output to C.

Running time: O(N)

Sorted arrays A and B

Total running time: Time taken to sort the arrays
+
Time taken to find common entries

= O(N log N)

Suppose N ~ 10^{10}, how much time does your computer take?

N log N ~ 40 * 10^{10} instructions **~ 40 seconds**

```
i ← 1, j ← 1
while i ≤ N and j ≤ N do
    if A[i] = B[j] then
        output A[i]
        i ← i + 1
        j ← j + 1
    else if A[i] < B[j] then
        i ← i + 1
    else
        j ← j + 1
end while
```

What is the need to study algorithms if computers were infinitely fast?

- Algorithm should halt and output the correct solution on each instance.
- Any correct method for solving a problem is good enough.

Computers are not infinitely fast!

- Computing time is a bounded resource.
- Storage space is another resource.

How to solve these problems?

- Algorithms using
  - ✓ Recursion
  - ✓ Divide and conquer
  - ✓ Dynamic programming
  - ✓ Greedy
- Reduction to an known problem
- Advanced techniques like combinatorial optimization, LP, randomization, advanced data structures, etc.

Analyzing time complexity

Exercise: For each function f(n) and time t in the following table, determine the largest size n of a problem that can be solved in time t, assuming that the algorithm to solve the problem takes f(n) microseconds.

	1 second (10^6 us)	1 minute (6*10^7 us)	1 hour (3.6 * 10^9 us)	1 day (8.64*10^9 us)
log n				
n				
n log n				
n^2				
n^3				
2^n				
n!				

Analyzing time complexity

Exercise: For each function f(n) and time t in the following table, determine the largest size n of a problem that can be solved in time t, assuming that the algorithm to solve the problem takes f(n) microseconds.

	1 second (10^6 us)	1 minute (6*10^7 us)	1 hour (3.6 * 10^9 us)	1 day (8.64*10^9 us)
log n	$2^{10^{\wedge}6}$			
n				
n log n				
n^2				
n^3				
2^n				
n!				

Analyzing time complexity

Exercise: For each function f(n) and time t in the following table, determine the largest size n of a problem that can be solved in time t, assuming that the algorithm to solve the problem takes f(n) microseconds.

	1 second (10^6 us)	1 minute (6*10^7 us)	1 hour (3.6 * 10^9 us)	1 day (8.64*10^9 us)
log n	2^{10^6}			
n	10^6			
n log n	~ 62,746			
n^2	~ 1,000			
n^3	~100			
2^n	~19			
n!	~9			

Analyzing time complexity

Exercise: For each function f(n) and time t in the following table, determine the largest size n of a problem that can be solved in time t, assuming that the algorithm to solve the problem takes f(n) microseconds.

	1 second (10^6 us)	1 minute (6*10^7 us)	1 hour (3.6 * 10^9 us)	1 day (8.64*10^9 us)
log n	$2^{10^{\wedge}6}$			$2^{8.64*10^{\wedge}9}$
n	10^6			
n log n	~62,746			
n^2	~1,000			
n^3	~100			
2^n	~19			~36
n!	~9			~16